ABSTRACT

The **Blood Bank Management System** is a database-driven application designed to efficiently manage blood donation, storage, and distribution in hospitals and blood banks. The system ensures the availability of safe and adequate blood supply by streamlining donor registrations, blood sample management, recipient requests, and transaction tracking.

This system automates critical processes such as **donor and recipient management, blood sample testing, hospital coordination, appointment scheduling, and real-time blood stock updates**. It integrates key stakeholders, including **donors, recipients, hospitals, administrators, and blood bank staff**, ensuring seamless communication and operational efficiency.

Design of Relational Schemas, Creation of Database and Tables for Blood Bank Management System

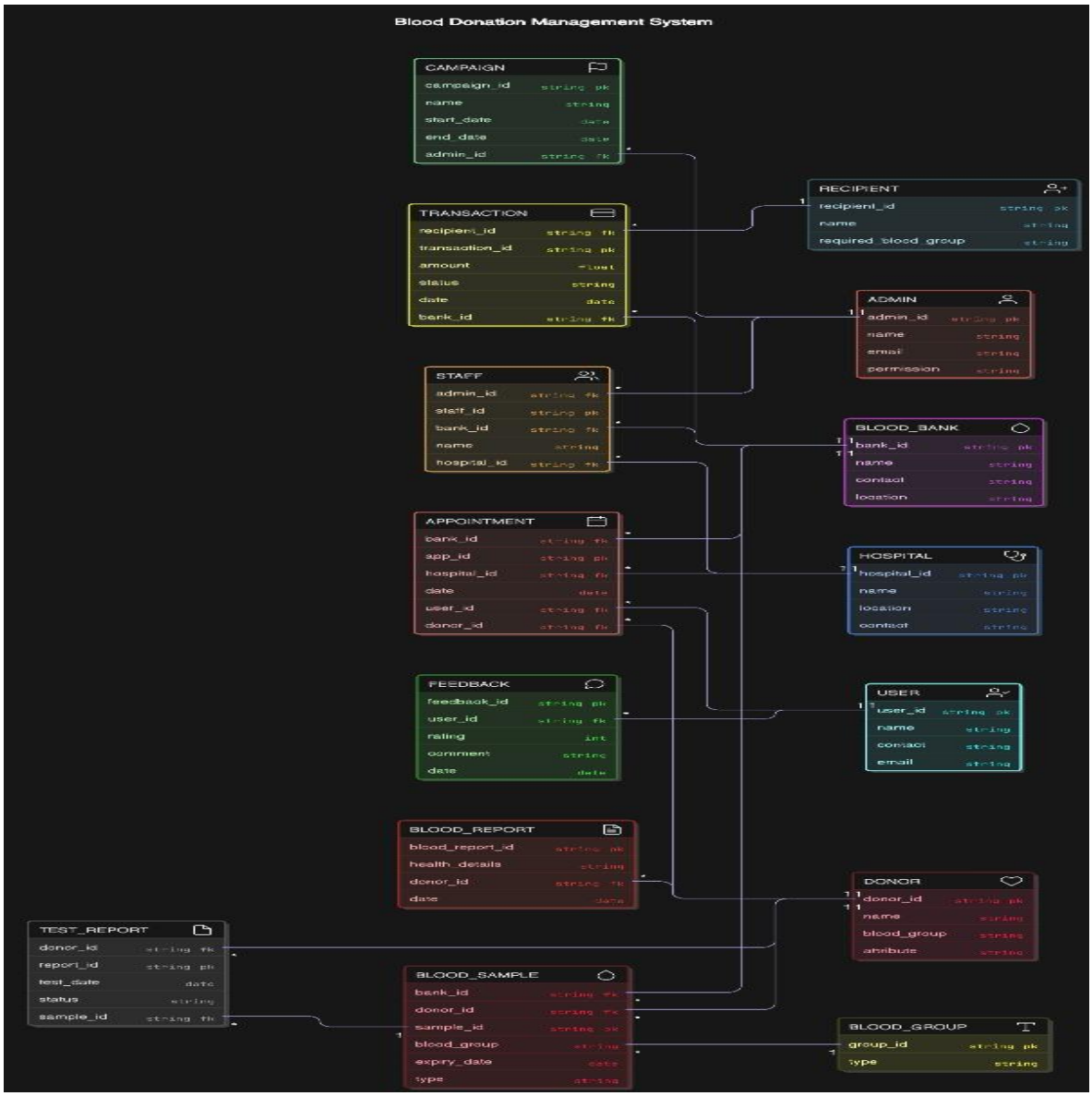


Figure 2.1: Relational Schema for blood bank management system

Creation of Database and Tables - DDL Commands

```
CREATE TABLE Admin (
admin_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100), email VARCHAR(100), contact VARCHAR(20), permissions TEXT
);
INSERT INTO Admin (name, email,permissions) VALUES
('Rahul Sharma', 'rahul.sharma@example.com', 'Full Access'),
('Ananya Gupta', 'ananya.gupta@example.com', 'Manage Campaigns, View Reports'),
('Vikram Singh', 'vikram.singh@example.com', 'Manage Donors, View Transactions'),
('Priya Mehta', 'priya.mehta@example.com', Manage Hospitals, View Feedback'),
('Amit Verma', 'amit.verma@example.com', "View Reports, Manage Blood Stock');
 SELECT * FROM  Admin;
```

Output

```
admin_id        name    email   permissions
1       Rahul Sharma    rahul.sharma@example.com        Full Access
2       Ananya Gupta    ananya.gupta@example.com        Manage Campaigns, View Reports
3       Vikram Singh    vikram.singh@example.com        Manage Donors, View Transactions
4       Priya Mehta     priya.mehta@example.com Manage Hospitals, View Feedback
5       Amit Verma      amit.verma@example.com  View Reports, Manage Blood Stock

[Execution complete with exit code 0]
```

```
CREATE TABLE CAMPAIGN (
    Campaign_id INT PRIMARY KEY,
```

```
    Admin_id INT,
    Name VARCHAR(100),
    Start_date DATE,
    End_date DATE,
    FOREIGN KEY (Admin_id) REFERENCES ADMIN(Admin_id)
);
INSERT INTO CAMPAIGN VALUES
(1, 1, 'Blood Drive 2025', '2025-01-10', '2025-01-15'),
(2, 2, 'Emergency Blood Collection', '2025-02-20', '2025-02-25'),
(3, 3, 'Save Lives Initiative', '2025-03-05', '2025-03-10'),
(4, 4, 'Red Cross Support', '2025-04-12', '2025-04-17'),
(5, 5, 'Annual Blood Donation Camp', '2025-05-01', '2025-05-07');
```

```
Campaign_id     Admin_id        Name    Start_date      End_date
1       1       Blood Drive 2025        2025-01-10      2025-01-15
2       2       Emergency Blood Collection      2025-02-20      2025-02-25
3       3       Save Lives Initiative   2025-03-05      2025-03-10
4       4       Red Cross Support       2025-04-12      2025-04-17
5       5       Annual Blood Donation Camp      2025-05-01      2025-05-07

[Execution complete with exit code 0]
```

```
CREATE TABLE HOSPITAL (
    Hospital_id INT PRIMARY KEY,
    Name VARCHAR(100),
    Location VARCHAR(100),
    Contact VARCHAR(50)
);
INSERT INTO HOSPITAL VALUES
(1, 'City Hospital', 'Downtown', '9876543210'),
(2, 'Green Valley Clinic', 'Uptown', '8765432109'),
(3, 'Health Care Center', 'West Side', '7654321098'),
(4, 'Sunrise Medical', 'East Side', '6543210987'),
(5, 'Metro Hospital', 'Central', '5432109876');
```

Output

```
Hospital_id     Name    Location        Contact
1       City Hospital   Downtown        9876543210
2       Green Valley Clinic     Uptown  8765432109
3       Health Care Center      West Side       7654321098
4       Sunrise Medical East Side       6543210987
5       Metro Hospital  Central 5432109876

[Execution complete with exit code 0]
```

```
CREATE TABLE BLOOD_BANK (
    Bank_id INT PRIMARY KEY,
    Name VARCHAR(100),
    Contact VARCHAR(50),
    Location VARCHAR(100)
);
INSERT INTO BLOOD_BANK VALUES
(1, 'Red Cross Bank', '1234567890', 'Main Street'),
(2, 'Life Saver Bank', '2345678901', 'City Center'),
(3, 'Donate & Save Bank', '3456789012', 'North Zone'),
(4, 'Hope Blood Bank', '4567890123', 'South Zone'),
(5, 'Universal Blood Bank', '5678901234', 'East Zone');
SELECT * FROM BLOOD_BANK;
```

Output

```
Bank_id Name    Contact Location
1       Red Cross Bank  1234567890      Main Street
2       Life Saver Bank 2345678901      City Center
3       Donate & Save Bank      3456789012      North Zone
4       Hope Blood Bank 4567890123      South Zone
5       Universal Blood Bank    5678901234      East Zone

[Execution complete with exit code 0]
```

```
CREATE TABLE STAFF (
    Staff_id INT PRIMARY KEY,
    Admin_id INT,
    Bank_id INT,
```

```
    Hospital_id INT,
    Name VARCHAR(100),
    FOREIGN KEY (Admin_id) REFERENCES ADMIN(Admin_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id),
    FOREIGN KEY (Hospital_id) REFERENCES HOSPITAL(Hospital_id)
);
INSERT INTO STAFF (Staff_id, Admin_id, Bank_id, Hospital_id, Name) VALUES
(1, 1, 1, 1, 'James Wilson'),
(2, 2, 2, 2, 'Olivia Martinez'),
(3, 3, 3, 3, 'Ethan Johnson'),
(4, 4, 4, 4, 'Sophia Hernandez'),
(5, 5, 5, 5, 'Daniel Brown');
```

Output

```
Staff_id        Admin_id        Bank_id Hospital_id     Name
1       1       1       1       James Wilson
2       2       2       2       Olivia Martinez
3       3       3       3       Ethan Johnson
4       4       4       4       Sophia Hernandez
5       5       5       5       Daniel Brown

[Execution complete with exit code 0]
```

```
CREATE TABLE DONOR (
    Donor_id INT PRIMARY KEY,
    Name VARCHAR(100),
    Blood_Group VARCHAR(10),
```

```
    Attribute VARCHAR(100)
);
INSERT INTO DONOR VALUES
(1, 'David White', 'A+', 'Regular Donor'),
(2, 'Jessica Blue', 'O-', 'First Time Donor'),
(3, 'Samuel Green', 'B+', 'Frequent Donor'),
(4, 'Sophia Black', 'AB-', 'Emergency Donor'),
(5, 'Ryan Grey', 'O+', 'Regular Donor');
```

Output

Donor_id	Name	Blood_Group	Attribute
1	David White	A+	Regular Donor
2	Jessica Blue	O-	First Time Donor
3	Samuel Green	B+	Frequent Donor
4	Sophia Black	AB-	Emergency Donor
5	Ryan Grey	O+	Regular Donor

[Execution complete with exit code 0]

```
CREATE TABLE USER (
    User_id INT PRIMARY KEY,
    Name VARCHAR(100),
    Contact VARCHAR(50),
    Email VARCHAR(100)
);
 INSERT INTO USER (User_id, Name, Contact, Email) VALUES
(1, 'Ava Johnson', '9876543210', 'ava.johnson@example.com'),
(2, 'Liam Smith', '8765432109', 'liam.smith@example.com'),
(3, 'Emma Brown', '7654321098', 'emma.brown@example.com'),
(4, 'Noah Williams', '6543210987', 'noah.williams@example.com'),
```

(5, 'Olivia Davis', '5432109876', 'olivia.davis@example.com');

Output

```
User_id Name    Contact Email
1       Ava Johnson     9876543210      ava.johnson@example.com
2       Liam Smith      8765432109      liam.smith@example.com
3       Emma Brown      7654321098      emma.brown@example.com
4       Noah Williams   6543210987      noah.williams@example.com
5       Olivia Davis    5432109876      olivia.davis@example.com

[Execution complete with exit code 0]
```

```
CREATE TABLE RECIPIENT (
    Recipient_id INT PRIMARY KEY,
    Name VARCHAR(100),
    Required_blood_group VARCHAR(10)
);
INSERT INTO RECIPIENT VALUES
(1, 'Tom Adams', 'A+'),
(2, 'Nancy Lee', 'O-'),
(3, 'Peter Parker', 'B+'),
(4, 'Bruce Wayne', 'AB-'),
(5, 'Clark Kent', 'O+');
```

Output

```
Recipient_id    Name    Required_blood_group
1       Tom Adams       A+
2       Nancy Lee       O-
3       Peter Parker    B+
4       Bruce Wayne     AB-
5       Clark Kent      O+

[Execution complete with exit code 0]
```

```
CREATE TABLE TRANSACTION (
    Transaction_id INT PRIMARY KEY,
    Recipient_id INT,
    Bank_id INT,
    Amount DECIMAL(10,2),
    Status VARCHAR(50),
    Date DATE,
    FOREIGN KEY (Recipient_id) REFERENCES RECIPIENT(Recipient_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id)
);
INSERT INTO TRANSACTION VALUES
(1, 1, 1, 500.00, 'Completed', '2025-01-12'),
(2, 2, 2, 300.00, 'Pending', '2025-02-21'),
(3, 3, 3, 450.00, 'Completed', '2025-03-07'),
(4, 4, 4, 600.00, 'Cancelled', '2025-04-14'),
(5, 5, 5, 350.00, 'Completed', '2025-05-03');
```

Output

```
Transaction_id  Recipient_id    Bank_id Amount  Status  Date
1       1       1       500.00  Completed       2025-01-12
2       2       2       300.00  Pending 2025-02-21
3       3       3       450.00  Completed       2025-03-07
4       4       4       600.00  Cancelled       2025-04-14
5       5       5       350.00  Completed       2025-05-03

[Execution complete with exit code 0]
```

```
CREATE TABLE APPOINTMENT (
    App_id INT PRIMARY KEY,
    Donor_id INT,
    User_id INT,
    Bank_id INT,
    Hospital_id INT,
    Date DATE,
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),
    FOREIGN KEY (User_id) REFERENCES USER(User_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id),
    FOREIGN KEY (Hospital_id) REFERENCES HOSPITAL(Hospital_id)
);
INSERT INTO APPOINTMENT (App_id, Donor_id, User_id, Bank_id, Hospital_id, Date)
VALUES
(1, 1, 1, 1, 1, '2025-02-20'),
(2, 2, 2, 2, 2, '2025-03-15'),
(3, 3, 3, 3, 3, '2025-04-10'),
(4, 4, 4, 4, 4, '2025-05-05'),
(5, 5, 5, 5, 5, '2025-06-01');
SELECT * FROM TRANSACTION;
```

Output

```
App_id  Donor_id        User_id Bank_id Hospital_id     Date
1       1       1       1       1       2025-02-20
2       2       2       2       2       2025-03-15
3       3       3       3       3       2025-04-10
4       4       4       4       4       2025-05-05
5       5       5       5       5       2025-06-01

[Execution complete with exit code 0]
```

```
CREATE TABLE APPOINTMENT (
    App_id INT PRIMARY KEY,
    Donor_id INT,
    User_id INT,
    Bank_id INT,
    Hospital_id INT,
    Date DATE,
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),
    FOREIGN KEY (User_id) REFERENCES USER(User_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id),
    FOREIGN KEY (Hospital_id) REFERENCES HOSPITAL(Hospital_id)
);

INSERT INTO APPOINTMENT VALUES
(1, 1, 1, 1, 1, '2025-02-20'),
(2, 2, 2, 2, 2, '2025-03-15'),
(3, 3, 3, 3, 3, '2025-04-10'),
(4, 4, 4, 4, 4, '2025-05-05'),
(5, 5, 5, 5, 5, '2025-06-01');
```

Output

```
App_id  Donor_id        User_id Bank_id Hospital_id     Date
1       1       1       1       1       2025-02-20
2       2       2       2       2       2025-03-15
3       3       3       3       3       2025-04-10
4       4       4       4       4       2025-05-05
5       5       5       5       5       2025-06-01

[Execution complete with exit code 0]
```

```
CREATE TABLE FEEDBACK (
    Feedback_id INT PRIMARY KEY,
    User_id INT,
    Rating INT,
    Comment TEXT,
    Date DATE,
    FOREIGN KEY (User_id) REFERENCES USER(User_id)
);
INSERT INTO FEEDBACK VALUES
(1, 1, 5, 'Great experience, very organized.', '2025-02-22'),
(2, 2, 4, 'Good service but needs improvement.', '2025-03-17'),
(3, 3, 5, 'Excellent staff and management.', '2025-04-12'),
(4, 4, 3, 'Average experience, long waiting time.', '2025-05-07'),
(5, 5, 5, 'Highly recommend this facility.', '2025-06-03');
```

Output

```
Feedback_id     User_id Rating  Comment Date
1       1       5       Great experience, very organized.       2025-02-22
2       2       4       Good service but needs improvement.     2025-03-17
3       3       5       Excellent staff and management. 2025-04-12
4       4       3       Average experience, long waiting time.  2025-05-07
5       5       5       Highly recommend this facility. 2025-06-03

[Execution complete with exit code 0]
```

```
CREATE TABLE BLOOD_REPORT (
    Blood_Report_id INT PRIMARY KEY,
    Donor_id INT,
    Health_Details TEXT,
    Date DATE,
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id)
);

INSERT INTO BLOOD_REPORT VALUES
(1, 1, 'Healthy donor, no infections.', '2025-02-21'),
(2, 2, 'Low hemoglobin, advised supplements.', '2025-03-16'),
(3, 3, 'Blood sugar levels normal.', '2025-04-11'),
(4, 4, 'Mild dehydration noted.', '2025-05-06'),
(5, 5, 'All health parameters are normal.', '2025-06-02');
```

Output

```
Blood_Report_id Donor_id        Health_Details  Date
1       1       Healthy donor, no infections.   2025-02-21
2       2       Low hemoglobin, advised supplements.    2025-03-16
3       3       Blood sugar levels normal.      2025-04-11
4       4       Mild dehydration noted. 2025-05-06
5       5       All health parameters are normal.       2025-06-02

[Execution complete with exit code 0]
```

```
CREATE TABLE TEST_REPORT (
    Report_id INT PRIMARY KEY,
    Donor_id INT,
    Sample_id INT,
    Test_date DATE,
    Status VARCHAR(50),
```

```
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),
    FOREIGN KEY (Sample_id) REFERENCES BLOOD_SAMPLE(Sample_id)
);

INSERT INTO TEST_REPORT VALUES
(1, 1, 1, '2025-02-22', 'Passed'),
(2, 2, 2, '2025-03-18', 'Failed - Low RBC count'),
(3, 3, 3, '2025-04-13', 'Passed'),
(4, 4, 4, '2025-05-08', 'Passed'),
(5, 5, 5, '2025-06-04', 'Failed - Infection detected');
```

Output

Report_id	Donor_id	Sample_id	Test_date	Status
1	1	1	2025-02-22	Passed
2	2	2	2025-03-18	Failed - Low RBC count
3	3	3	2025-04-13	Passed
4	4	4	2025-05-08	Passed
5	5	5	2025-06-04	Failed - Infection detected

[Execution complete with exit code 0]

```
CREATE TABLE BLOOD_SAMPLE (
    Sample_id INT PRIMARY KEY,
    Donor_id INT,
    Bank_id INT,
    Blood_group VARCHAR(10),
    Expiry_date DATE,
    Type VARCHAR(50),
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id)
);
```

```
INSERT INTO BLOOD_SAMPLE VALUES
(1, 1, 1, 'A+', '2025-05-22', 'Whole Blood'),
(2, 2, 2, 'O-', '2025-06-18', 'Plasma'),
(3, 3, 3, 'B+', '2025-07-13', 'Red Blood Cells'),
(4, 4, 4, 'AB-', '2025-08-08', 'Platelets'),
(5, 5, 5, 'O+', '2025-09-04', 'Plasma');
```

Output

```
Sample_id       Donor_id        Bank_id Blood_group     Expiry_date     Type
1       1       1       A+      2025-05-22      Whole Blood
2       2       2       O-      2025-06-18      Plasma
3       3       3       B+      2025-07-13      Red Blood Cells
4       4       4       AB-     2025-08-08      Platelets
5       5       5       O+      2025-09-04      Plasma

[Execution complete with exit code 0]
```

```
CREATE TABLE MANAGES (
   Admin_id INT,
   Campaign_id INT,
   PRIMARY KEY (Admin_id, Campaign_id),
   FOREIGN KEY (Admin_id) REFERENCES ADMIN(Admin_id),
   FOREIGN KEY (Campaign_id) REFERENCES CAMPAIGN(Campaign_id)
);
-- Insert Data into MANAGES
INSERT INTO MANAGES VALUES
(1, 101),
(2, 102),
(3, 103),
(4, 104),
(5, 105);
```

Output

```
Admin_id        Campaign_id
1       101
2       102
3       103
4       104
5       105

[Execution complete with exit code 0]
```

```
-- Creating TRANSACTION Table
CREATE TABLE TRANSACTION (
    Transaction_id INT PRIMARY KEY,
    Recipient_id INT,
    Bank_id INT,
    Amount DECIMAL(10,2),
    Status VARCHAR(50),
    Date DATE,
    FOREIGN KEY (Recipient_id) REFERENCES RECIPIENT(Recipient_id) ON DELETE CASCADE,
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id) ON DELETE CASCADE
);

-- Inserting Data into TRANSACTION (After RECIPIENT and BLOOD_BANK)
INSERT INTO TRANSACTION VALUES
(1, 1, 1, 500.00, 'Completed', '2025-02-10'),
(2, 2, 2, 1000.00, 'Pending', '2025-03-12'),
(3, 3, 3, 750.00, 'Failed', '2025-04-14'),
(4, 4, 4, 1200.00, 'Completed', '2025-05-16')
```

(5, 5, 5, 300.00, 'Pending', '2025-06-18');

Output

```
Transaction_id  Recipient_id    Bank_id Amount  Status  Date
1       1       1       500.00  Completed       2025-02-10
2       2       2       1000.00 Pending 2025-03-12
3       3       3       750.00  Failed  2025-04-14
4       4       4       1200.00 Completed       2025-05-16
5       5       5       300.00  Pending 2025-06-18

[Execution complete with exit code 0]
```

```
-- Creating APPOINTMENT Table
CREATE TABLE APPOINTMENT (
    App_id INT PRIMARY KEY,
    Donor_id INT,
    User_id INT,
    Bank_id INT,
    Hospital_id INT,
    Date DATE,
    FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),
    FOREIGN KEY (User_id) REFERENCES USER(User_id),
    FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id),
    FOREIGN KEY (Hospital_id) REFERENCES HOSPITAL(Hospital_id)
);

-- Inserting Data into APPOINTMENT
INSERT INTO APPOINTMENT VALUES
(1, 1, 1, 1, 1, '2025-02-15'),
(2, 2, 2, 2, 2, '2025-03-20'),
(3, 3, 3, 3, 3, '2025-04-25'),
```

(4, 4, 4, 4, 4, '2025-05-30'),

(5, 5, 5, 5, 5, '2025-06-10');

Output

```
App_id  Donor_id        User_id Bank_id Hospital_id     Date
1       1       1       1       1       2025-02-15
2       2       2       2       2       2025-03-20
3       3       3       3       3       2025-04-25
4       4       4       4       4       2025-05-30
5       5       5       5       5       2025-06-10

[Execution complete with exit code 0]
```

-- Creating BLOOD_SAMPLE Table

CREATE TABLE BLOOD_SAMPLE (

Sample_id INT PRIMARY KEY,

Donor_id INT,

Bank_id INT,

Blood_group VARCHAR(10),

Quantity INT, -- Added Quantity column

Expiry_date DATE,

Type VARCHAR(50),

FOREIGN KEY (Donor_id) REFERENCES DONOR(Donor_id),

FOREIGN KEY (Bank_id) REFERENCES BLOOD_BANK(Bank_id)

);

-- Inserting Data into BLOOD_SAMPLE

INSERT INTO BLOOD_SAMPLE (Sample_id, Donor_id, Bank_id, Blood_group, Quantity, Expiry_date, Type) VALUES

(1, 1, 1, 'A+', 500, '2025-06-01', 'Whole Blood'),

(2, 2, 2, 'O-', 300, '2025-06-05', 'Plasma'),

(3, 3, 3, 'B+', 450, '2025-06-10', 'Platelets'),

(4, 4, 4, 'AB-', 600, '2025-06-15', 'Red Blood Cells'),

(5, 5, 5, 'O+', 350, '2025-06-20', 'Whole Blood');

Output

```
Sample_id       Donor_id        Bank_id Blood_group     Quantity        Expiry_date
1       1       1       A+      500     2025-06-01      Whole Blood
2       2       2       O-      300     2025-06-05      Plasma
3       3       3       B+      450     2025-06-10      Platelets
4       4       4       AB-     600     2025-06-15      Red Blood Cells
5       5       5       O+      350     2025-06-20      Whole Blood

[Execution complete with exit code 0]
```

TRIGGERS

1. DELIMITER $$

CREATE TRIGGER trg_format_admin_name

BEFORE INSERT ON Admin

FOR EACH ROW

BEGIN

SET NEW.name = UPPER(NEW.name);

END$$

DELIMITER ;

INSERT INTO Admin (name, email, contact, permissions)

VALUES ('Sneha Rao', 'sneha.rao@example.com', '9871234567', 'View Reports');

SELECT * FROM Admin WHERE email = 'sneha.rao@example.com';

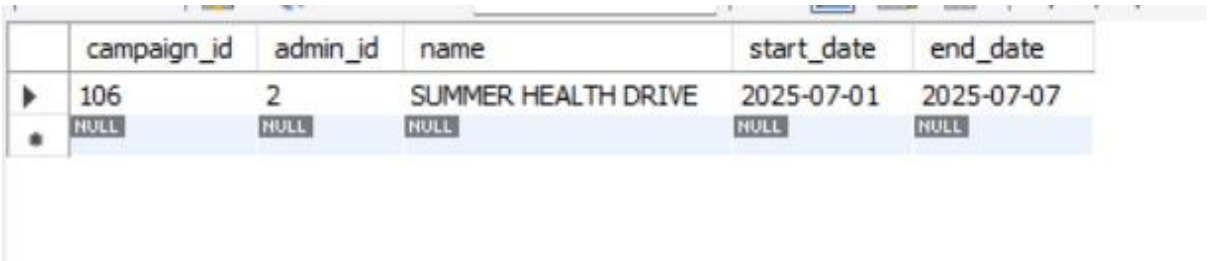
```
DELIMITER $$

CREATE TRIGGER trg_format_admin_name
BEFORE INSERT ON Admin
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END$$

DELIMITER ;

INSERT INTO Admin (name, email, contact, permissions)
VALUES ('Sneha Rao', 'sneha.rao@example.com', '9871234567', 'View Reports');

SELECT * FROM Admin WHERE email = 'sneha.rao@example.com';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

admin_id	name	email	contact	permissions
3450	Sneha Rao	sneha.rao@example.com	9871234567	View Reports
3453	SNEHA RAO	sneha.rao@example.com	9871234567	View Reports
NULL	NULL	NULL	NULL	NULL

2. DELIMITER $$

```
CREATE TRIGGER trg_format_campaign_name
BEFORE INSERT ON Campaign
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END$$

DELIMITER ;

INSERT INTO Campaign VALUES (106, 2, 'summer health drive', '2025-07-01', '2025-07-07');
SELECT * FROM Campaign WHERE campaign_id = 106;
```



campaign_id	admin_id	name	start_date	end_date
106	2	SUMMER HEALTH DRIVE	2025-07-01	2025-07-07
NULL	NULL	NULL	NULL	NULL

3.
```
DELIMITER $$
CREATE TRIGGER trg_format_hospital_data
BEFORE INSERT ON Hospital
FOR EACH ROW
BEGIN
    -- Format hospital name to uppercase
    SET NEW.name = UPPER(NEW.name);

    -- Check if contact number is 10 digits
    IF CHAR_LENGTH(NEW.contact) != 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Contact number must be exactly 10 digits';
    END IF;
END$$

DELIMITER ;

INSERT INTO Hospital (hospital_id, name, location, contact)
VALUES (6, 'Lotus Care', 'North Side', '9876543210');
SELECT * FROM Hospital;
```

```
DELIMITER $$

CREATE TRIGGER trg_format_hospital_data
BEFORE INSERT ON Hospital
FOR EACH ROW
BEGIN
    -- Format hospital name to uppercase
    SET NEW.name = UPPER(NEW.name);

    -- Check if contact number is 10 digits
    IF CHAR_LENGTH(NEW.contact) != 10 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Contact number must be exactly 10 digits';
    END IF;
END$$

DELIMITER ;
INSERT INTO Hospital (hospital_id, name, location, contact)
VALUES (6, 'Lotus Care', 'North Side', '9876543210');

SELECT * FROM Hospital;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

hospital_id	name	location	contact
1	City Hospital	Downtown	9876543210
2	Green Valley Clinic	Uptown	8765432109
3	Health Care Center	West Side	7654321098
4	Sunrise Medical	East Side	6543210987
5	Metro Hospital	Central	5432109876
6	LOTUS CARE	North Side	9876543210
NULL	NULL	NULL	NULL

4. DELIMITER $$

```
CREATE TRIGGER trg_format_blood_bank_name

BEFORE INSERT ON Blood_Bank

FOR EACH ROW

BEGIN

    SET NEW.name = UPPER(NEW.name);

END$$


DELIMITER ;

INSERT INTO Blood_Bank (bank_id, name, contact, location)

VALUES (6, 'life savers bank', '9123456789', 'North Wing');SELECT * FROM
Blood_Bank WHERE bank_id = 6;
```

```
DELIMITER $$

CREATE TRIGGER trg_format_blood_bank_name
BEFORE INSERT ON Blood_Bank
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END$$

DELIMITER ;
INSERT INTO Blood_Bank (bank_id, name, contact, location)
VALUES (6, 'life savers bank', '9123456789', 'North Wing');

SELECT * FROM Blood_Bank WHERE bank_id = 6;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

bank_id	name	contact	location
6	LIFE SAVERS BANK	9123456789	North Wing
NULL	NULL	NULL	NULL

5. DELIMITER $$

CREATE TRIGGER trg_format_staff_name

BEFORE INSERT ON Staff

FOR EACH ROW

BEGIN

    SET NEW.name = UPPER(NEW.name);

END$$

DELIMITER ;

INSERT INTO Staff (staff_id, admin_id, bank_id, hospital_id, name)

VALUES (6, 1, 1, 1, 'alex carter');

SELECT * FROM Staff WHERE staff_id = 6;

```
DELIMITER $$

CREATE TRIGGER trg_format_staff_name
BEFORE INSERT ON Staff
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END$$

DELIMITER ;
INSERT INTO Staff (staff_id, admin_id, bank_id, hospital_id, name)
VALUES (6, 1, 1, 1, 'alex carter');

SELECT * FROM Staff WHERE staff_id = 6;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

staff_id	admin_id	bank_id	hospital_id	name
6	1	1	1	ALEX CARTER
NULL	NULL	NULL	NULL	NULL

6.

```
DELIMITER $$

CREATE TRIGGER trg_format_donor_namedonor

BEFORE INSERT ON Donor

FOR EACH ROW

BEGIN

   SET NEW.name = UPPER(NEW.name);

END$$
```

DELIMITER ;

INSERT INTO Donor (donor_id, name, blood_group, attribute)

VALUES (6, 'maria fernandez', 'B-', 'First Time Donor');

SELECT * FROM Donor WHERE donor_id = 6;

```
DELIMITER $$

CREATE TRIGGER trg_format_donor_name
BEFORE INSERT ON Donor
FOR EACH ROW
BEGIN
    SET NEW.name = UPPER(NEW.name);
END$$

DELIMITER ;
INSERT INTO Donor (donor_id, name, blood_group, attribute)
VALUES (6, 'maria fernandez', 'B-', 'First Time Donor');

SELECT * FROM Donor WHERE donor_id = 6;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

donor_id	name	blood_group	attribute
6	MARIA FERNANDEZ	B-	First Time Donor
NULL	NULL	NULL	NULL

VIEWS

1. CREATE VIEW user_info_view_with_join AS

SELECT

u.user_id,

u.name AS user_name,

u.email,

u.contact

FROM

User u

JOIN

Appointment a ON u.user_id = a.user_id;

SELECT * FROM user_info_view_with_join;

```
CREATE VIEW user_info_view_with_join AS
SELECT
    u.user_id,
    u.name AS user_name,
    u.email,
    u.contact
FROM
    User u
JOIN
    Appointment a ON u.user_id = a.user_id;

SELECT * FROM user_info_view_with_join;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	user_name	email	contact
1	Ava Johnson	ava.johnson@example.com	9876543210
2	Liam Smith	liam.smith@example.com	8765432109
3	Emma Brown	emma.brown@example.com	7654321098
4	Noah Williams	noah.williams@example.com	6543210987
5	Olivia Davis	olivia.davis@example.com	5432109876

2. CREATE VIEW recipient_info_view_with_transaction AS

SELECT

r.recipient_id,

r.name AS recipient_name,

r.required_blood_group,

t.transaction_id,

t.amount,

t.status,

t.date,

b.name AS bank_name

FROM

Recipient r

JOIN

Transaction t ON r.recipient_id = t.recipient_id

JOIN

Blood_Bank b ON t.bank_id = b.bank_id;

SELECT * FROM recipient_info_view_with_transaction;

```
CREATE VIEW recipient_info_view_with_transaction AS
SELECT
    r.recipient_id,
    r.name AS recipient_name,
    r.required_blood_group,
    t.transaction_id,
    t.amount,
    t.status,
    t.date,
    b.name AS bank_name
FROM
    Recipient r
JOIN
    Transaction t ON r.recipient_id = t.recipient_id
JOIN
    Blood_Bank b ON t.bank_id = b.bank_id;

SELECT * FROM recipient_info_view_with_transaction;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

recipient_id	recipient_name	required_blood_group	transaction_id	amount	status	date	bank_name
1	Tom Adams	A+	1	500.00	Completed	2025-02-10	Red Cross Bank
2	Nancy Lee	O-	2	1000.00	Pending	2025-03-12	Life Saver Bank
3	Peter Parker	B+	3	750.00	Failed	2025-04-14	Donate & Save Bank
4	Bruce Wayne	AB-	4	1200.00	Completed	2025-05-16	Hope Blood Bank
5	Clark Kent	O+	5	300.00	Pending	2025-06-18	Universal Blood Bank

3.

```
CREATE VIEW transaction_details_view AS
SELECT
    t.transaction_id,
    r.name AS recipient_name,
    r.required_blood_group,
    b.name AS blood_bank_name,
    t.amount,
    t.status,
    t.date
FROM
    Transaction t
JOIN
    Recipient r ON t.recipient_id = r.recipient_id
JOIN
    Blood_Bank b ON t.bank_id = b.bank_id;
SELECT * FROM transaction_details_view;
```

```
CREATE VIEW transaction_details_view AS
SELECT
    t.transaction_id,
    r.name AS recipient_name,
    r.required_blood_group,
    b.name AS blood_bank_name,
    t.amount,
    t.status,
    t.date
FROM
    Transaction t
JOIN
    Recipient r ON t.recipient_id = r.recipient_id
JOIN
    Blood_Bank b ON t.bank_id = b.bank_id;

SELECT * FROM transaction_details_view;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

transaction_id	recipient_name	required_blood_group	blood_bank_name	amount	status	date
1	Tom Adams	A+	Red Cross Bank	500.00	Completed	2025-02-10
2	Nancy Lee	O-	Life Saver Bank	1000.00	Pending	2025-03-12
3	Peter Parker	B+	Donate & Save Bank	750.00	Failed	2025-04-14
4	Bruce Wayne	AB-	Hope Blood Bank	1200.00	Completed	2025-05-16
5	Clark Kent	O+	Universal Blood Bank	300.00	Pending	2025-06-18

4.

```
CREATE VIEW appointment_details_view AS
SELECT
    a.app_id,
    d.name AS donor_name,
    u.name AS user_name,
    b.name AS blood_bank_name,
    h.name AS hospital_name,
    a.date AS appointment_date
FROM
    Appointment a
JOIN
    Donor d ON a.donor_id = d.donor_id
JOIN
    User u ON a.user_id = u.user_id
JOIN
    Blood_Bank b ON a.bank_id = b.bank_id
JOIN
    Hospital h ON a.hospital_id = h.hospital_id;

SELECT * FROM appointment_details_view;
```

```sql
CREATE VIEW appointment_details_view AS
SELECT
    a.app_id,
    d.name AS donor_name,
    u.name AS user_name,
    b.name AS blood_bank_name,
    h.name AS hospital_name,
    a.date AS appointment_date
FROM
    Appointment a
JOIN
    Donor d ON a.donor_id = d.donor_id
JOIN
    User u ON a.user_id = u.user_id
JOIN
    Blood_Bank b ON a.bank_id = b.bank_id
JOIN
    Hospital h ON a.hospital_id = h.hospital_id;

SELECT * FROM appointment_details_view;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

app_id	donor_name	user_name	blood_bank_name	hospital_name	appointment_date
1	David White	Ava Johnson	Red Cross Bank	City Hospital	2025-02-15
2	Jessica Blue	Liam Smith	Life Saver Bank	Green Valley Clinic	2025-03-20
3	Samuel Green	Emma Brown	Donate & Save Bank	Health Care Center	2025-04-25
4	Sophia Black	Noah Williams	Hope Blood Bank	Sunrise Medical	2025-05-30
5	Ryan Grey	Olivia Davis	Universal Blood Bank	Metro Hospital	2025-06-10

5.

```sql
CREATE VIEW user_feedback_view AS
SELECT
    f.feedback_id,
    u.user_id,
    u.name AS user_name,
    u.email,
    f.rating,
```

f.comment,

f.date AS feedback_date

FROM

Feedback f

JOIN

User u ON f.user_id = u.user_id;

SELECT * FROM user_feedback_view;

```sql
CREATE VIEW user_feedback_view AS
SELECT
    f.feedback_id,
    u.user_id,
    u.name AS user_name,
    u.email,
    f.rating,
    f.comment,
    f.date AS feedback_date
FROM
    Feedback f
JOIN
    User u ON f.user_id = u.user_id;

SELECT * FROM user_feedback_view;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

feedback_id	user_id	user_name	email	rating	comment	feedback_date
1	1	Ava Johnson	ava.johnson@example.com	5	Great experience, very organized.	2025-02-22
2	2	Liam Smith	liam.smith@example.com	4	Good service but needs improvement.	2025-03-17
3	3	Emma Brown	emma.brown@example.com	5	Excellent staff and management.	2025-04-12
4	4	Noah Williams	noah.williams@example.com	3	Average experience, long waiting time.	2025-05-07
5	5	Olivia Davis	olivia.davis@example.com	5	Highly recommend this facility.	2025-06-03

6. CREATE VIEW donor_blood_report_view AS

SELECT

br.blood_report_id,

d.donor_id,

d.name AS donor_name,

d.blood_group,

br.health_details,

br.date AS report_date

FROM

Blood_Report br

JOIN

Donor d ON br.donor_id = d.donor_id;

SELECT * FROM donor_blood_report_view;

```
CREATE VIEW donor_blood_report_view AS
SELECT
    br.blood_report_id,
    d.donor_id,
    d.name AS donor_name,
    d.blood_group,
    br.health_details,
    br.date AS report_date
FROM
    Blood_Report br
JOIN
    Donor d ON br.donor_id = d.donor_id;

SELECT * FROM donor_blood_report_view;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

blood_report_id	donor_id	donor_name	blood_group	health_details	report_date
1	1	David White	A+	Healthy donor, no infections.	2025-02-21
2	2	Jessica Blue	O-	Low hemoglobin, advised supplements.	2025-03-16
3	3	Samuel Green	B+	Blood sugar levels normal.	2025-04-11
4	4	Sophia Black	AB-	Mild dehydration noted.	2025-05-06
5	5	Ryan Grey	O+	All health parameters are normal.	2025-06-02

CURSORS

1.

```
DELIMITER $$
CREATE PROCEDURE SimpleAppointmentCursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE app_id INT;
    DECLARE app_date DATE;

    DECLARE cur CURSOR FOR
        SELECT app_id, date FROM Appointment;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO app_id, app_date;

        IF done THEN
            LEAVE read_loop;
        END IF;

        SELECT CONCAT('Appointment ID: ', app_id, ', Date: ', app_date) AS Info;
    END LOOP;
    CLOSE cur;
END$$
DELIMITER ;
```

CALL SimpleAppointmentCursor();SELECT CONCAT_WS(', ', 'Appointment ID: ' , app_id, 'Date: ', date) AS Info FROM Appointment;

```
DELIMITER $$

CREATE PROCEDURE SimpleAppointmentCursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE app_id INT;
    DECLARE app_date DATE;

    DECLARE cur CURSOR FOR
        SELECT app_id, date FROM Appointment;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO app_id, app_date;

        IF done THEN
            LEAVE read_loop;
        END IF;

        SELECT CONCAT('Appointment ID: ', app_id, ', Date: ', app_date) AS Info;
    END LOOP;

    CLOSE cur;
END$$

DELIMITER ;

CALL SimpleAppointmentCursor();
SELECT CONCAT_WS(', ', 'Appointment ID: ' , app_id, 'Date: ', date) AS Info FROM Appointment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Info
Appointment ID: , 1, Date: , 2025-02-15
Appointment ID: , 2, Date: , 2025-03-20
Appointment ID: , 3, Date: , 2025-04-25
Appointment ID: , 4, Date: , 2025-05-30
Appointment ID: , 5, Date: , 2025-06-10

2. DELIMITER //

```
CREATE PROCEDURE fetch_blood_samples()

BEGIN

    DECLARE done INT DEFAULT FALSE;
    DECLARE sample_id INT;
    DECLARE blood_type VARCHAR(10);
    DECLARE sample_cursor CURSOR FOR
        SELECT sample_id, blood_type FROM Blood_Sample;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN sample_cursor;
    read_loop: LOOP
        FETCH sample_cursor INTO sample_id, blood_type;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SELECT CONCAT('Sample ID: ', sample_id, ', Blood Type: ', blood_type) AS
Info;
    END LOOP;
    CLOSE sample_cursor;

END;

DELIMITER ;
CALL fetch_blood_samples();
SELECT CONCAT('Sample ID: ', sample_id) AS Info FROM Blood_Sample;
```

```
DELIMITER //

CREATE PROCEDURE fetch_blood_samples()
BEGIN
    -- Declare variables to hold column values
    DECLARE done INT DEFAULT FALSE;
    DECLARE sample_id INT;
    DECLARE blood_type VARCHAR(10);

    -- Declare the cursor
    DECLARE sample_cursor CURSOR FOR
        SELECT sample_id, blood_type FROM Blood_Sample;

    -- Declare exit handler for when there are no more rows
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Open the cursor
    OPEN sample_cursor;

    read_loop: LOOP
        FETCH sample_cursor INTO sample_id, blood_type;

        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Display the fetched data
        SELECT CONCAT('Sample ID: ', sample_id, ', Blood Type: ', blood_type) AS Info;
    END LOOP;

    -- Close the cursor
    CLOSE sample_cursor;
END;
//

DELIMITER ;

CALL fetch_blood_samples();
SELECT CONCAT('Sample ID: ', sample_id) AS Info FROM Blood_Sample;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

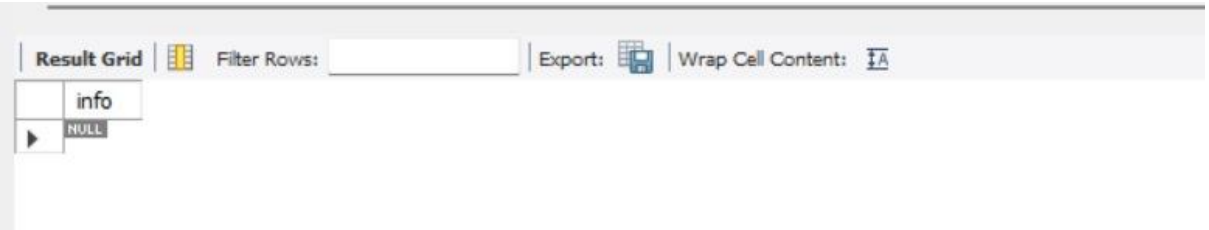
Info
Sample ID: 1
Sample ID: 2
Sample ID: 3
Sample ID: 4
Sample ID: 5

3.

```
 DELIMITER //
CREATE PROCEDURE simple_test_report_cursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE report_id INT;
    DECLARE test_type VARCHAR(100);

    DECLARE cur CURSOR FOR
        SELECT report_id, test_type FROM test_report;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur;
```

```
    read_loop: LOOP
        FETCH cur INTO report_id, test_type;
        IF done THEN
            LEAVE read_loop;
        END IF;

        SELECT CONCAT('Report ID: ', report_id, ', Test Type: ', test_type) AS info;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

CALL simple_test_report_cursor();
```


```
DELIMITER //

CREATE PROCEDURE simple_test_report_cursor()
BEGIN
    -- Declare variables
    DECLARE done INT DEFAULT FALSE;
    DECLARE report_id INT;
    DECLARE test_type VARCHAR(100);

    -- Declare the cursor
    DECLARE cur CURSOR FOR
        SELECT report_id, test_type FROM test_report;

    -- Handler to exit the loop
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Open cursor
    OPEN cur;

    -- Loop through rows
    read_loop: LOOP
        FETCH cur INTO report_id, test_type;
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Output fetched data
        SELECT CONCAT('Report ID: ', report_id, ', Test Type: ', test_type) AS info;
    END LOOP;

    -- Close cursor
    CLOSE cur;
END //

DELIMITER ;

CALL simple_test_report_cursor();
```



4.

```
DELIMITER $$

CREATE PROCEDURE SimpleTransactionCursor()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE trans_id INT;



   DECLARE cur CURSOR FOR
      SELECT trans_id FROM Transaction;

   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
   -- Open the cursor
   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO trans_id;
      IF done THEN
         LEAVE read_loop;
      END IF;

      SELECT CONCAT('Processing Transaction ID: ', trans_id);
   END LOOP;
   CLOSE cur;
```

END$$

DELIMITER ;

CALL SimpleTransactionCursor();

```
DELIMITER $$

CREATE PROCEDURE SimpleTransactionCursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE trans_id INT;

    -- Declare the cursor to select transaction IDs
    DECLARE cur CURSOR FOR
        SELECT trans_id FROM Transaction;

    -- Handler to set 'done' to true when no more rows are found
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Open the cursor
    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO trans_id;
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Action: for now, just print the transaction ID
        SELECT CONCAT('Processing Transaction ID: ', trans_id);
    END LOOP;

    -- Close the cursor
    CLOSE cur;
END$$

DELIMITER ;
CALL SimpleTransactionCursor();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONCAT('Processing Transaction ID: ', trans_id)
NULL

5.

DELIMITER $$

CREATE PROCEDURE FeedbackCursor()

BEGIN

DECLARE done INT DEFAULT FALSE;

```
    DECLARE feedback_id INT;
    DECLARE cur CURSOR FOR
    SELECT feedback_id FROM Feedback;
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
 OPEN cur;

    read_loop: LOOP
        FETCH cur INTO feedback_id;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SELECT CONCAT('Reading Feedback ID: ', feedback_id);
    END LOOP;

    CLOSE cur;
END$$
DELIMITER ;
CALL FeedbackCursor();
```

```
DELIMITER $$

CREATE PROCEDURE FeedbackCursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE feedback_id INT;

    -- Declare the cursor to select feedback IDs
    DECLARE cur CURSOR FOR
        SELECT feedback_id FROM Feedback;

    -- Handler to signal the end of the cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Open the cursor
    OPEN cur;

    read_loop: LOOP
        FETCH cur INTO feedback_id;
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Action: just printing/logging the feedback ID
        SELECT CONCAT('Reading Feedback ID: ', feedback_id);
    END LOOP;

    -- Close the cursor
    CLOSE cur;
END$$

DELIMITER ;
CALL FeedbackCursor();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CONCAT('Reading Feedback ID: ', feedback_id)
NULL

SETS

1.

SELECT donor_id AS person_id, 'Donor' AS role

FROM Appointment

WHERE hospital_id = 1

UNION

-- Users with appointments at Blood Bank 1

SELECT user_id AS person_id, 'User' AS role

FROM Appointment

WHERE bank_id = 1;

```
-- Donors with appointments at Hospital 1
SELECT donor_id AS person_id, 'Donor' AS role
FROM Appointment
WHERE hospital_id = 1

UNION

-- Users with appointments at Blood Bank 1
SELECT user_id AS person_id, 'User' AS role
FROM Appointment
WHERE bank_id = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

person_id	role
1	Donor
1	User

2.

SELECT admin_id AS person_id, 'Blood Bank Admin' AS role

FROM Admin

WHERE assigned_bank_id = 1

UNION

SELECT admin_id AS person_id, 'Campaign Admin' AS role

FROM Admin

WHERE assigned_campaign_id = 1;

```
-- Example using correct column names
SELECT admin_id AS person_id, 'Blood Bank Admin' AS role
FROM Admin
WHERE assigned_bank_id = 1

UNION

SELECT admin_id AS person_id, 'Campaign Admin' AS role
FROM Admin
WHERE assigned_campaign_id = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

admin_id	name	email	contact	permissions
1	Rahul Sharma	rahul.sharma@example.com	9990000001	Full Access
2	Ananya Gupta	ananya.gupta@example.com	9990000002	Manage Campaigns, View Reports
3	Vikram Singh	vikram.singh@example.com	9990000003	Manage Donors, View Transactions
4	baibhav	priya.mehta@example.com	9990000004	Manage Hospitals, View Feedback
5	Amit Verma	amit.verma@example.com	9990000005	View Reports, Manage Blood Stock
3447	Jatin	harrsh@gmail.com	NULL	NULL
3448	vivek	viv@gmail.com	NULL	NULL
3449	Neha Reddy	neha.reddy@example.com	9876543210	Full Access
3450	Sneha Rao	sneha.rao@example.com	9871234567	View Reports
3453	SNEHA RAO	sneha.rao@example.com	9871234567	View Reports

admin 1

3.

SELECT sample_id, 'Group A+' AS category

FROM Blood_Sample

WHERE blood_group = 'A+'

UNION

-- Blood Samples stored in Blood Bank 1

SELECT sample_id, 'Stored at Blood Bank 1' AS category

FROM Blood_Sample

WHERE bank_id = 1;

```
-- Blood Samples of blood group 'A+'
SELECT sample_id, 'Group A+' AS category
FROM Blood_Sample
WHERE blood_group = 'A+'

UNION

-- Blood Samples stored in Blood Bank 1
SELECT sample_id, 'Stored at Blood Bank 1' AS category
FROM Blood_Sample
WHERE bank_id = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sample_id	category
1	Group A+
1	Stored at Blood Bank 1

4.

SELECT admin_id FROM Manages WHERE campaign_id % 2 = 0 -- Even campaign IDs

UNION

SELECT admin_id FROM Manages WHERE campaign_id % 2 = 1; -- Odd campaign IDs

```
SELECT admin_id FROM Manages WHERE campaign_id % 2 = 0  -- Even campaign IDs
UNION
SELECT admin_id FROM Manages WHERE campaign_id % 2 = 1; -- Odd campaign IDs
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

admin_id
2
4
1
3
5

5.

SELECT donor_id FROM Blood_Sample WHERE type = 'Whole Blood'

UNION

SELECT donor_id FROM Blood_Sample WHERE type = 'Plasma';

```
SELECT donor_id FROM Blood_Sample WHERE type = 'Whole Blood'
UNION
SELECT donor_id FROM Blood_Sample WHERE type = 'Plasma';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

donor_id
1
5
2

BloodSync Life

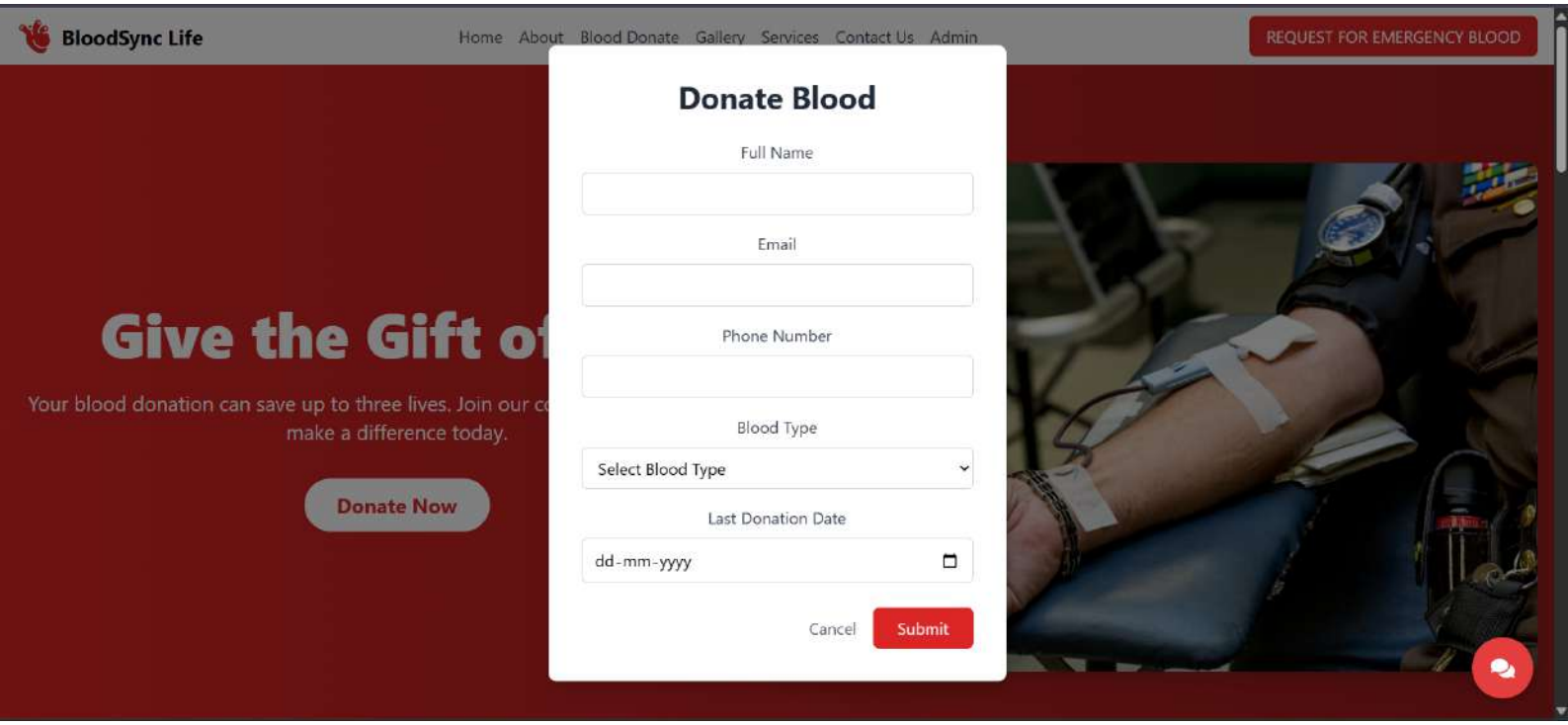
Home About Blood Donate Gallery Services Contact Us Admin

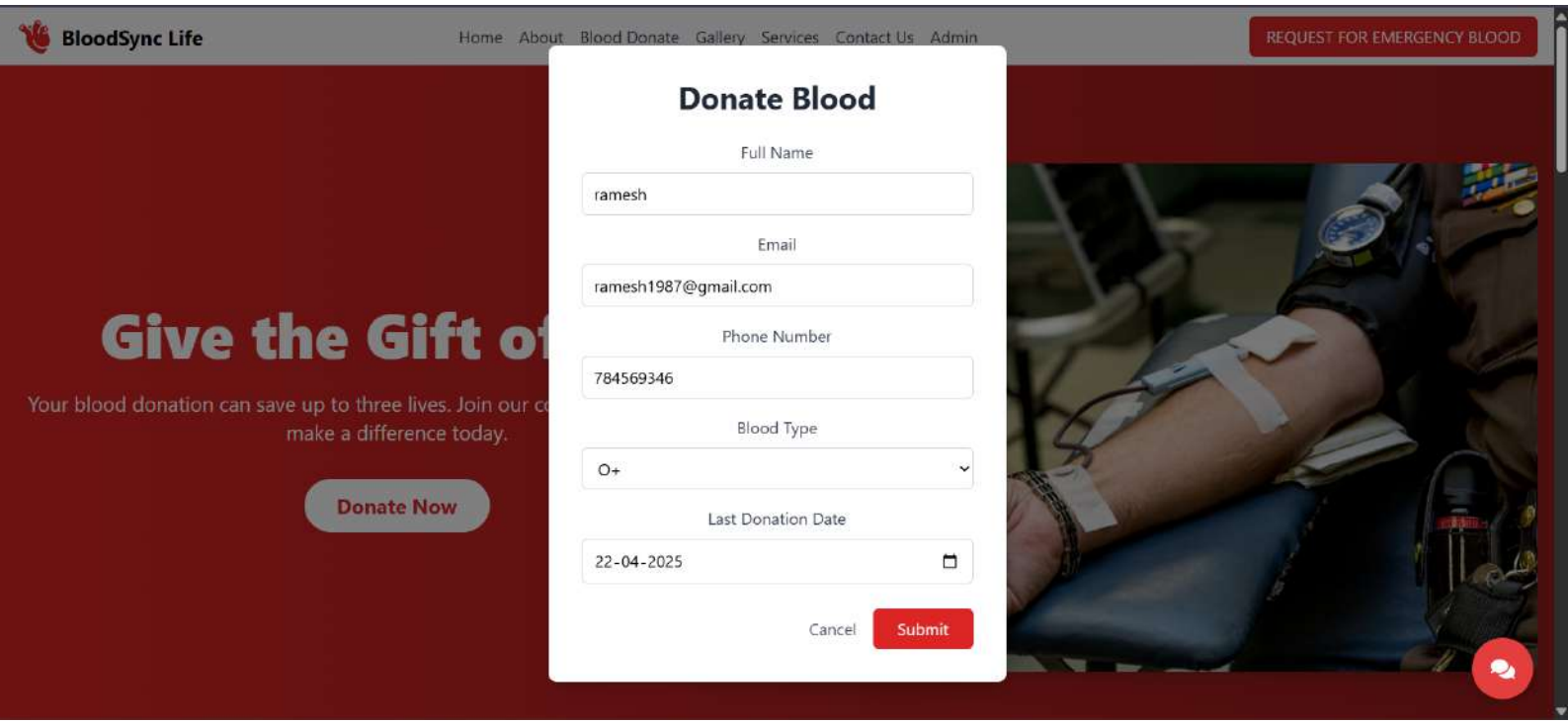
REQUEST FOR EMERGENCY BLOOD

Give the Gift of Life

Your blood donation can save up to three lives. Join our community of heroes and make a difference today.

Donate Now


BloodSync Life
Home
About
Blood Donate
Gallery
Services
Contact Us
Admin
REQUEST FOR EMERGENCY BLOOD
Give the Gift of
Your blood donation can save up to three lives. Join our c
make a difference today.
Donate Now
Donate Blood
Full Name
Email
Phone Number
Blood Type
Select Blood Type
Last Donation Date
dd-mm-yyyy
Cancel
Submit


BloodSync Life
Home About Blood Donate Gallery Services Contact Us Admin
REQUEST FOR EMERGENCY BLOOD
Give the Gift of
Your blood donation can save up to three lives. Join our c
make a difference today.
Donate Now
Donate Blood
Full Name
ramesh
Email
ramesh1987@gmail.com
Phone Number
784569346
Blood Type
O+
Last Donation Date
22-04-2025
Cancel
Submit

MySQL 8.0 Command Line Cli

```
|  6 | NULL     | ravi@gmail.com                | 768433267    | NULL     | B+             | NULL           | NULL
       | Ravi        | 2025-04-17 00:00:00 |
+----+----------+-------------------------------+--------------+----------+----------------+----------------+-----------
-------+-------------+---------------------+
5 rows in set (0.04 sec)

mysql> select * from blood_donor;
+----+----------+-------------------------------+--------------+----------+----------------+----------------+-----------
-------+-------------+---------------------+
| id | userName | email                         | phone        | imageURL | bloodDonorType | additionalInfo | location_p
incode | fullName    | lastDonation        |
+----+----------+-------------------------------+--------------+----------+----------------+----------------+-----------
-------+-------------+---------------------+
|  1 | NULL     | john.xavier.example@gmail.com | 987654321    | NULL     | B+             | NULL           | NULL
       | John Xavier | 2025-03-10 00:00:00 |
|  2 | NULL     | chandra.xyz.example@gmail.com | 456787654    | NULL     | O+             | NULL           | NULL
       | Chandra     | 2025-03-08 00:00:00 |
|  4 | NULL     | anjany.example@gmail.com      | 787654567654 | NULL     | O-             | NULL           | NULL
       | Anjany      | 2025-03-01 00:00:00 |
|  5 | NULL     | testuser02@example.com        | 12345677654  | NULL     | O+             | NULL           | NULL
       | testuser2   | 2025-04-15 00:00:00 |
|  6 | NULL     | ravi@gmail.com                | 768433267    | NULL     | B+             | NULL           | NULL
       | Ravi        | 2025-04-17 00:00:00 |
|  7 | NULL     | ramesh1987@gmail.com          | 784569346    | NULL     | O+             | NULL           | NULL
       | ramesh      | 2025-04-22 00:00:00 |
+----+----------+-------------------------------+--------------+----------+----------------+----------------+-----------
-------+-------------+---------------------+
6 rows in set (0.00 sec)

mysql>
```

```
127.0.0.1 - - [17/Apr/2025 12:31:00] "OPTIONS /api/blood-donations HTTP/1.1" 200 -
*** Server Side Data Received *** {'name': 'ramesh', 'email': 'ramesh1987@gmail.com',
'phone': '784569346', 'bloodType': 'O+', 'lastDonation': '2025-04-22'}
2025-04-17 12:31:01,003 INFO sqlalchemy.engine.Engine SELECT DATABASE()
2025-04-17 12:31:01,003 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-04-17 12:31:01,007 INFO sqlalchemy.engine.Engine SELECT @@sql_mode
2025-04-17 12:31:01,007 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-04-17 12:31:01,009 INFO sqlalchemy.engine.Engine SELECT @@lower_case_table_names
2025-04-17 12:31:01,009 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-04-17 12:31:01,013 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-04-17 12:31:01,017 INFO sqlalchemy.engine.Engine INSERT INTO blood_donor (`userNa
me`, `fullName`, email, phone, `imageURL`, `bloodDonorType`, `lastDonation`, `addition
alInfo`, location_pincode) VALUES (%(userName)s, %(fullName)s, %(email)s, %(phone)s, %
(imageURL)s, %(bloodDonorType)s, %(lastDonation)s, %(additionalInfo)s, %(location_pinc
ode)s)
2025-04-17 12:31:01,019 INFO sqlalchemy.engine.Engine [generated in 0.00194s] {'userNa
me': None, 'fullName': 'ramesh', 'email': 'ramesh1987@gmail.com', 'phone': '784569346'
, 'imageURL': None, 'bloodDonorType': 'O+', 'lastDonation': '2025-04-22', 'additionalI
nfo': None, 'location_pincode': None}
2025-04-17 12:31:01,029 INFO sqlalchemy.engine.Engine COMMIT
127.0.0.1 - - [17/Apr/2025 12:31:01] "POST /api/blood-donations HTTP/1.1" 201 -
```